Using Neural Networks for Security Attack Detection

UE24CS352A Machine Learning - Mini-Project | Tanisha Khoria (PES2UG24CS551), Sparsha Arun (PES2UG24CS513) | PES University

1. Problem Statement

Network intrusions (denial of service, scanning, remote and privilege-escalation attacks) must be detected from connection-level traffic records. We reproduce the neural network of *Using Neural Network for Security Attacks Detection* (CS229, Stanford, 2021) on NSL-KDD, and extend it with a tuned MLP and a Random Forest baseline.

2. Dataset

NSL-KDD (KDD'99 without duplicate records): KDDTrain+ (125,973 records) and KDDTest+ (22,544), 41 features each (3 categorical: protocol, service, flag). Attacks fall into DoS, Probe, R2L and U2R. The data is heavily imbalanced and the test set contains 17 attack types absent from training, which makes generalisation hard.

Class	Normal	DoS	Probe	R2L	U2R
Train	67,343	45,927	11,656	995	52
Test	9,711	7,460	2,421	2,752	200

3. Approach

Part 1 - paper reproduction. One-hot encoding of protocol/service/flag (122 features), standardisation to zero mean and unit variance (fit on train). Targets are 24 classes: normal, the 22 attack types seen in training, and an 'other' class for unseen attacks. Network 122-500-24 with sigmoid hidden units and softmax output, mini-batch SGD (lr 0.5, batch 1000, L2 1e-4), 10 epochs, as in the paper. Because the paper reports strong dependence on random initialization, we repeat training with 10 seeds. Besides 24-class accuracy we compute Normal/Abnormal accuracy from the same predictions. **Part 2 - extensions.** Min-Max scaled inputs, a ReLU MLP (122-128-64, Adam, early stopping) and a 200-tree Random Forest, evaluated on 5-class (Normal/DoS/Probe/R2L/U2R) and binary tasks.

4. Implementation Overview

Python (pandas, scikit-learn, matplotlib). `src/download_data.py` fetches the data; `src/run_experiment.py` runs preprocessing, both experiments and writes metrics and figures to `results/`. Code: <GitHub repo link>.

5. Results

Experiment	Metric	Result
Paper network (10 seeds)	24-class test acc	68.6% +/- 0.1 (paper: ~70%)
Paper network (10 seeds)	Normal/Abnormal test acc	75.2% +/- 0.5 (range 74.3-76.3; paper: ~80%)
Paper network	Train acc (24-class)	~98% (paper: almost 100%)
ReLU MLP / Random Forest	5-class acc | macro-F1	78.5% | 0.567 / 75.4% | 0.492
ReLU MLP / Random Forest	Binary acc	81.9% / 78.4%

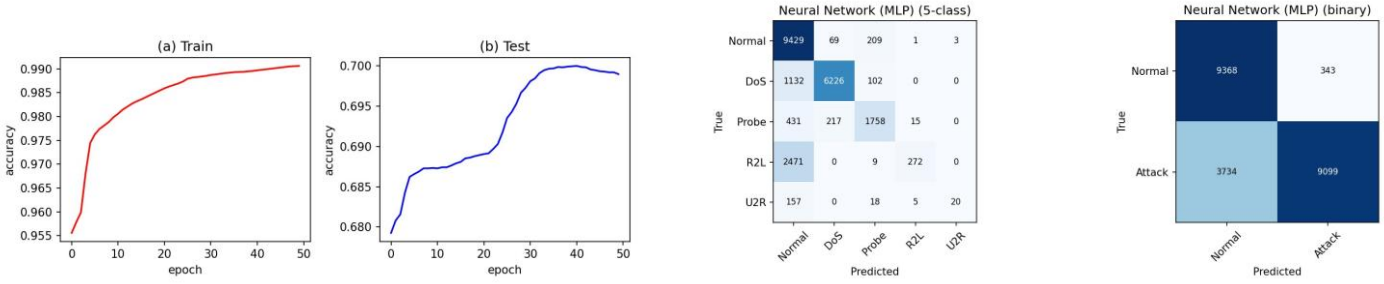


Fig. 1: Paper network accuracy vs epoch (train/test, seed 0); ReLU MLP confusion matrices (5-class, binary).

6. Conclusions

Our reproduction matches the paper's headline behavior: near-perfect fit on training data but only about 69% 24-class test accuracy, since 17 test attack types never appear in training. Normal/Abnormal accuracy is higher (75%), though below the paper's 80%; we did not see the paper's occasional runs above 85% in 10 seeds, and our run-to-run spread is small. Test accuracy keeps creeping up slowly with more epochs (about 70% by epoch 50), unlike the paper's description of it fluctuating after epoch 10. Our ReLU MLP beats Random Forest (+3.1 points on 5-class, +3.5 on binary) and gives higher macro-F1, but both models miss most R2L and U2R attacks (recall about 10% for the MLP): unseen attacks resemble normal traffic, so Normal is over-predicted. Future work: class weighting or oversampling for R2L/U2R, feature selection, and evaluation on newer datasets such as CIC-IDS2017.